

1. Authentication

Login

- **Endpoint:** POST /login/
- **Description:** Authenticates a user (student or professor) and returns an access token.
- **Request Body (JSON):**

JSON

```
{
  "email": "user@example.com",
  "password": "mypassword"
}
```

- **Response (200 OK):**

JSON

```
{
  "token": "abcdef1234567890",
  "is_staff": true
}
```

- **Error (401 / 400):** Returns an error message if the credentials are missing or invalid.
-

2. Professor Endpoints (is_staff=True required)

These endpoints are used for creating exams, uploading materials, and managing participants.

Retrieve or Create Exams

- **Endpoint:** GET /exams/ | POST /exams/
- **GET:** Lists all exams created by the requesting professor.
- **POST (Create Exam):**
 - **Body (JSON):** title, date (ISO 8601 String), description (optional), duration_minutes (optional, Default: 60).
 - **Response (201 Created):** Returns the ID and details of the newly created exam.

Professor Dashboard

- **Endpoint:** GET /professor/dashboard/
- **Description:** Provides an overview of all the professor's exams, including participant counts, the number of generated questions, and a list of enrolled students.

Upload Exam Material & Generate Questions (PDF)

- **Endpoint:** POST /exams/<exam_id>/upload-pdf/

- **Description:** Accepts a PDF lecture file, extracts the text, and automatically generates multiple-choice questions using NLP.
- **Request:** multipart/form-data with the key pdf_file.
- **Response (200 OK):** Confirmation message including the number of generated questions.

Manually Upload Exam Questions (JSON)

- **Endpoint:** POST /exams/<exam_id>/upload-questions/
- **Description:** As an alternative to the PDF generation, ready-made questions can be uploaded directly as a JSON file.
- **Request:** multipart/form-data with the key exam_file.

Review & Retrieve Exam Questions

- **Endpoint:** GET /exams/<exam_id>/questions/professor/
- **Description:** Loads the generated or uploaded questions from the file system so the professor can review them.

Save Exam Questions (Finalize)

- **Endpoint:** POST /exams/<exam_id>/questions/save/
- **Description:** Saves the finalized questions after review.
- **Request Body (JSON):**

JSON

```
{
  "questions": [
    { "text": "...", "options": ["A", "B", "C", "D"], "answer": "A" }
  ]
}
```

Manage Enrollment List (Enrollments)

- **Endpoints:**
 - * GET /exams/<exam_id>/enrollments/ (View the enrollment list)
 - POST /exams/<exam_id>/enrollments/ (Enroll students via matriculation number; Body: matriculation_numbers as a list)
 - PUT / DELETE /exams/<exam_id>/enrollments/<matrikel>/ (Update or remove a specific student's enrollment)

3. Identity Verification (Computer Vision)

Verify Identity

- **Endpoint:** POST /verify-identity/
- **Description:** Compares a live webcam image with a photo ID (utilizes DeepFace with the RetinaFace detector and VGG-Face model).
- **Request:** multipart/form-data

- live_image: The live photo of the student.
 - id_image: The photo of the ID card.
- **Response (200 OK):**

JSON

```
{
  "verified": true,
  "distance": 0.45,
  "threshold": 0.80,
  "face_detected_live": true,
  "face_detected_id": true,
  "message": "✅ Identität bestätigt.",
  "hints": []
}
```

Note: If distance > 0.80, verified will be false and practical improvement tips (hints) will be returned.

4. Student Endpoints

List Exams

- **Endpoint:** GET /exams/
- **Description:** Lists all available exams in the system for students.

Join Exam

- **Endpoint:** POST /exams/<exam_id>/join/
- **Description:** Checks if the student's matriculation number is on the exam's enrollment list (ExamEnrollment). If it is, an ExamParticipation record is created.
- **Error (403 Forbidden):** Returned if the student is not on the enrollment list.

Retrieve Exam Questions (Exam Execution)

- **Endpoint:** GET /exams/<exam_id>/questions/
- **Description:** Retrieves the final exam questions for the student to answer.
- **Requirement:** The student must have successfully joined the exam via the /join/ endpoint first.

Request Manual Check (Help Request)

- **Endpoint:** POST /help-request/
- **Description:** If automated identity verification repeatedly fails, the student can request a manual review from the provider/professor.
- **Request Body (JSON):**

JSON

```
{  
  "exam_id": 1,  
  "message": "My ID is not recognized due to heavy glare."  
}
```

5. Additional Defined Routes (Learning & RAG System)

Your `urls.py` also defines additional endpoints for a *Learning Feature* and an external *RAG Generation*. The logic for these is located in `learning_views.py` and `rag_views.py` (which are not in the provided code snippet), but they exist as valid API routes:

- **Learning Material & Plans:**
 - GET `/learn/slides/` & POST `/learn/upload/` (Lecture slides)
 - GET `/learn/plans/` & GET `/learn/plans/<plan_id>/` (Learning paths)
 - POST `/learn/slides/<slide_id>/create-plan/` (Generate a learning plan from a slide)
- **Flashcards:**
 - GET `/learn/topics/<topic_id>/flashcards/`
 - POST `/learn/topics/<topic_id>/flashcards/generate/`
 - POST `/learn/flashcards/<card_id>/mark/` (Mark learning status)
- **Alternative RAG Question Generation:**
 - POST `/exams/<exam_id>/rag-generate/` (Likely triggers the pipeline defined in `advanced_rag.py`)